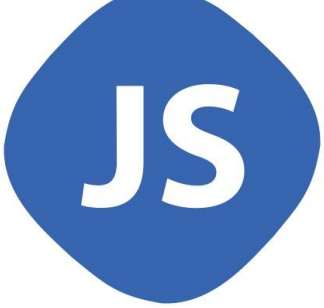


Модули

- Введение
- Модульная система JS
- Экспорт/импорт переменных и функций
- Особенности модулей
- Особенности браузеров
- Немного практики

Введение

Долгое время в JavaScript отсутствовал синтаксис модулей на уровне языка. Это не было проблемой, первые скрипты были маленькими и простыми. В модулях не было необходимости.

По мере роста функционала – растет объем программного кода приложения. Соответственно возникает желание разделить его на много файлов, так называемых **модулей**. Модуль обычно содержит **класс** или **библиотеку с функциями**.

> ***Примечание***. Классы мы еще не проходили, поэтому работу модулей разберу на примере работы **библиотеки функций**.

Приведу цитату самого себя.

> ***Примечание***. Чтобы основной файл был избавлен от излишнего нагромождения программных конструкций, объявления функций принято выносить в отдельные файлы (впоследствии объединять в **модули**).
>
> Тема 6. Функции / Урок 1. Введение в функции JS.



Со временем скрипты становятся всё более сложными, поэтому сообщество придумало несколько вариантов организации кода в модули. Появились библиотеки для динамической подгрузки модулей.

Система модулей появилась в стандарте JavaScript в 2015 году и постепенно эволюционировала. На данный момент она поддерживается большинством браузеров.

При работе с модулями очень важно не забывать, что:

> ***Важно***. Модули не работают локально. Только через протокол **HTTP**(s). Если вы попытаетесь открыть веб-страницу локально, через протокол **file**://, вы обнаружите, что директивы **import**/**export** не работают.

На всякий случай: протокол **file** – это когда вы **открываете файл** двойным кликом (или через контекстное меню). Протокол **HTTP** – это когда вы **набираете в адресной строке** URL файла. Но для этого нужен локальный сервер.

Для тестирования приложений на своих курсах я использую локальный сервер **Open Server Panel** (но могут быть варианты).

> ***Примечание***. Локальный сервер вам понадобится в любом случае. Цель курса – программирование **асинхронных** приложений, а там без сервера никуда.

> ***Примечание***. Вы можете смело пропустить этот урок при первом ознакомлении с языком программирования JS. Но при наличии планов изучения фреймворков JS или сервера Node.js ознакомление с модульной системой обязательно.

Модульная система JS

Модуль – это просто файл. Один скрипт – это один модуль.

Модули могут загружать друг друга и использовать директивы **export** и **import**, чтобы обмениваться функциональностью, вызывать функции одного модуля из другого:

- **export** отмечает переменные и функции, которые **должны быть доступны** вне текущего файла (модуля).
- **import** позволяет **импортировать** функциональность из других файлов (модулей).

Начну с простых примеров. Например, пусть наш файл **storage.js** **экспортирует** функцию:

example_1. **storage.js**

```
// отдаем (экспортируем) во внешний файл функцию
export let fnSayHello = (name) => {
    alert ("hello, " + name + "!")
}
```

В файле **example_1.html импортируем** и используем эту функцию. Директива **import** загружает модуль по пути **./storage.js** относительно текущего файла и записывает экспортированную функцию **fnSayHello** в соответствующую переменную.

example_1. **example_1.html**

```
<script type="module">
    // импортируем функцию из модуля storage.js
    import { fnSayHello } from "./storage.js";
    // вызов импортированной из модуля функции
    fnSayHello("Lady Jane");
```

```
</script>
```

Необходимо явно указать браузеру, что **скрипт является модулем**, делается это при помощи атрибута <**script type="module"**>.

Браузер автоматически загрузит и запустит импортированный модуль (и те, модули, которые он сам импортирует, если будут соответствующие инструкции импорта), а затем запустит скрипт.

> ***Еще раз***. Чтобы в браузере работал режим **import/export**, для скрипта должен быть указан атрибут **type**: **type="module"**.

Надо понимать, что признак модульного скрипта не возможность импорта и экспорта, а атрибут **type**. Именно атрибут добавляет возможность операций **import**/**export**, а не наоборот.

Далеко не каждая функция из модуля должна быть экспортирована. Функция может использоваться внутри модуля в логике работы вашего приложения, т.е. быть **вспомогательной** при выполнении определенного спектра задач. Например, так, как приведено в демонстрационном примере **example_2**.

example_2. **storage.js**

```
// отдаем (экспортируем) во внешний файл функцию
// функция-выражение
export let fnSayHello = (name) => {
    name = fnUpperCase (name);
    alert ("Hello, " + name + "!");
}
// вспомогательная функция модуля
// функция-инструкция
function fnUpperCase (name) {
    let uName = name.toUpperCase();
```

```
    return uName;
}
```

example_2. **example_2.html**

```
<script type="module">
    // импортируем функцию из модуля storage.js
    import { fnSayHello } from "./storage.js";
    // вызов импортированной из модуля функции
    fnSayHello("Lady Jane");
</script>
```

Если возникнет необходимость использовать вспомогательную функцию во внешнем файле, не забудьте ее также **экспортировать** / **импортировать** соответственно в **каждом** из файлов.

example_3. **storage.js**

```
// отдаем (экспортируем) во внешний файл функцию
// функция-выражение
export let fnSayHello = (name) => {
    name = fnUpperCase (name);
    alert ("Hello, " + name + "!");
}
// экспортируем вспомогательную функцию модуля
// функция-инструкция
export function fnUpperCase (name) {
    name = name.toUpperCase() + "!";
    return name;
}
```



Функции модуля могут быть **функции-инструкции** или **функции-выражения**, это не принципиально.

example_3. **example_3.html**

```
<script type="module">
    // импортируем функции из модуля storage.js
    import { fnSayHello, fnUpperCase } from "./storage.js";
    // вызов функций из модуля
    fnSayHello("Lady Jane");
    alert (fnUpperCase('Hello, my dear User'));
</script>
```

Модульный скрипт может быть:

- **внешним**
- **встроенным (внутренним)**

Следующий пример демонстрирует поведение **внешнего** и **встроенного** модульного скрипта.

example_4. **example_4.html**

```
<!--
подключаем внешний модульный скрипт
такие скрипты без проблем взаимодействуют с деревом DOM
-->
<script type="module" src="./outer.js"></script>
<!--
встроенный модульный скрипт
содержит подключаемые функции
-->
```



```
<script type="module">
    // импортируем функцию из модуля storage.js
    import { fnSayHello } from "./storage.js";
    // вызов импортированной из модуля функции
    fnSayHello("Привет, это снова я...");
</script>
```

example_4. outer.js

```
// определение функции-выражения
let fnSayHi = () => {
    // вывод сообщения в диалог
    alert ("Я внешний скрипт!");
    // вывод сообщения в консоль
    console.log("Я внешний скрипт!");
    // вывод сообщения в документ
    document.body.insertAdjacentHTML("beforeend", "<h2>Я внешний
    скрипт!</h2>");
}
// вызов функции
fnSayHi();
```

example_4. storage.js

```
// отдаем (экспортируем) во внешний файл функцию
// функция-выражение
export let fnSayHello = (name) => {
    name = fnUpperCase (name);
```

```
    alert ("Hello, " + name + "!");
}

// вспомогательная функция модуля

// функция-инструкция

function fnUpperCase (name) {

    name = name.toUpperCase();

    return name;

}
```

Это было введение. А теперь **подробности**.

Экспорт/импорт переменных и функций

Export / import каждой переменной / функции

Поставьте **export** перед тем, что вы хотите экспортировать. Такой способ позволит импортировать во внешний файл каждое из экспортируемых значений.

Пример использования ключевого слова **export**:

```
// константа pi / экспортируем

export const pi = Math.PI;

// функция расчета квадрата числа / не экспортируем

const fnSquare = (x) => {

     return x * x;

};

// функция расчета площади круга / экспортируем
```

```
export let fnAreaCircle = (r) => {
    return pi * fnSquare(r);
};
```

Просто укажите **export** перед чем угодно, и это можно будет импортировать в другой файл.

example_5. mathematic.js

```
// константа pi / экспортируем
export const pi = Math.PI;
// квадрат числа / НЕ экспортируем
const fnSquare = (x) => {
    return x * x;
};
// куб числа / НЕ экспортируем
var fnCube = (x) => {
    return x * x * x;
};
// площадь круга / экспортируем
export let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};
// площадь треугольника / экспортируем
export function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
```

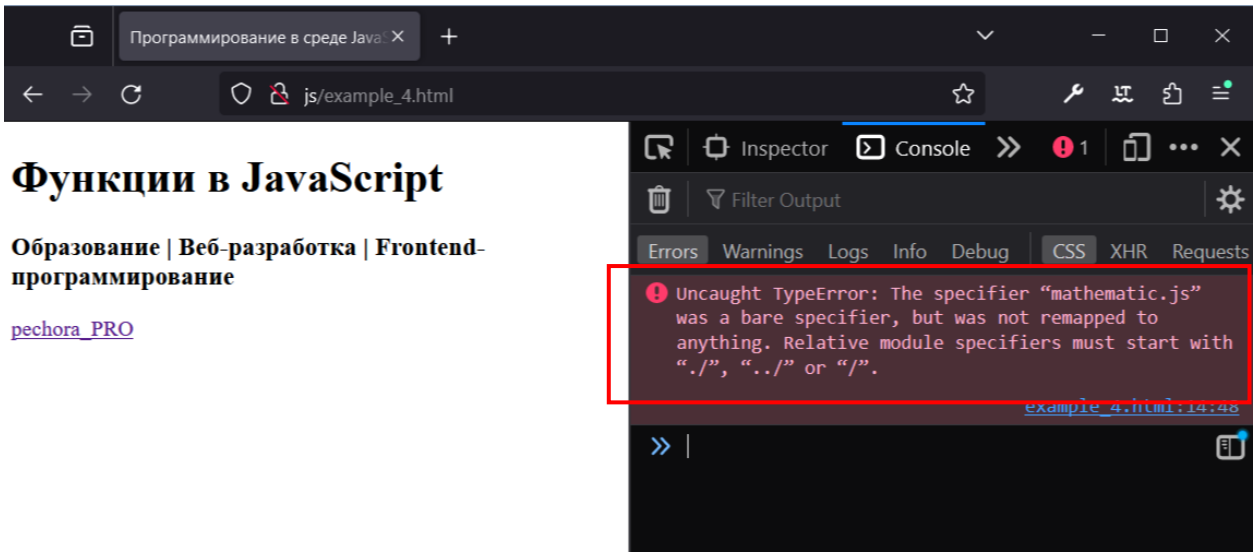
Для того чтобы импортировать переменные и функции, необходимо указать их имена в **фигурных скобках**. Ключевое слово **import**, затем список того, что вы хотите импортировать, это может быть одно или более значений, затем название модуля

Пример:

```
// импортируем перечисленные значения модуля
import { pi, fnAreaCircle } from "./mathematic.js";
// используем значения модуля
console.log(pi);
console.log(fnAreaCircle(3));
```

Путь './mathematic.js' означает, что модуль расположен в **текущей директории**.

> ***Важно***. Относительные спецификаторы пути при подключении модуля должны начинаться с "**./**", "**../**" или "**/**".



Если мы хотим импортировать что-то еще, можно просто добавить название переменной или функции в список:

```
// импортируем значения модуля
```

```
import { pi, fnAreaCircle, fnCube } from "./mathematic.js";
```

example_5. **example_5.html**

```
<script type="module">

    // импортируем только функции (без переменной)

    import { fnAreaCircle, fnAreaTriangle } from
    "./mathematic.js";

    let out = 'Площадь круга: ' + fnAreaCircle(4);

    out += '<p>Площадь треугольника: ' + fnAreaTriangle(4, 2);

    let target = document.getElementById("target");

    target.innerHTML = out;

</script>
```

Export / import нескольких значений сразу

Еще один способ экспорта — это **экспортирование сразу нескольких значений**:

```
// ...

// функция расчета площади треугольника

function fnAreaTriangle (a, h) {

     return (a * h) / 2;

};

// экспорт нескольких значений в конце модуля

export { pi, fnAreaCircle, fnAreaTriangle }
```

Или так:

```
// экспорт нескольких значений в конце модуля

export { pi }

export { fnAreaCircle, fnAreaTriangle }
```

example_6. mathematic.js

```
// константа pi
const pi = Math.PI;
// квадрат числа
const fnSquare = (x) => {
    return x * x;
};
// куб числа
var fnCube = (x) => {
    return x * x * x;
};
// площадь круга
let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};
// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
// экспорт нескольких значений
export { pi, fnAreaCircle, fnAreaTriangle }
```

Импорт переменных и функций **при этом не изменится**.

example_6. **example_6.html**

```
<script type="module">
    // импортируем только функции (без переменной)
    import { pi, fnAreaCircle, fnAreaTriangle } from
    "./mathematic.js";
    let out = 'Число PI: ' + pi;
    out += '<p>Площадь круга: ' + fnAreaCircle(4);
    out += '<p>Площадь треугольника: ' + fnAreaTriangle(4, 2);
    let target = document.getElementById("target");
    target.insertAdjacentHTML('afterbegin', out);
</script>
```

Export / import всего модуля

Но можно **импортировать все содержимое** модуля сразу (без указания списка значений):

```
// импортируем все содержимое модуля
import * as math from "./mathematic.js";
```

Это значит: "импортировать весь модуль и назвать его **math** в текущем модуле". При этом к импортированным сущностям обращение происходит через **math**, например так, **math.fnAreaCircle**.

example_7. **module/mathematic.js**

```
// константа pi
const pi = Math.PI;
// квадрат числа
const fnSquare = (x) => {
    return x * x;
```

```
};
// куб числа
var fnCube = (x) => {
    return x * x * x;
};
// площадь круга
let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};
// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
// экспорт нескольких значений
export { pi, fnAreaCircle, fnAreaTriangle }
```

example_7. example_7.html

```
<script type="module">
    // импортируем все содержимое модуля
    // при этом необходимо указать алиасное имя (as math)
    import * as math from "./module/mathematic.js";
    let target = document.getElementById("target");
    let out = 'Число PI: ' + math.pi;
    target.insertAdjacentHTML('afterbegin', out);
```

```
    out = '<p>Площадь круга: ' + math.fnAreaCircle(4);
    target.insertAdjacentHTML('afterbegin', out);
    out = '<p>Площадь треугольника: ' + math.fnAreaTriangle(4,
    2);
    target.insertAdjacentHTML('afterbegin', out);
</script>
```

Export / import по умолчанию

Часто требуется экспортировать из модуля что-то одно. Существует специальный механизм, который называется **экспорт по умолчанию**, и вы можете экспортировать с помощью него только **что-то одно**.

```
// ...
// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
// экспорт по умолчанию
export default fnAreaCircle;
```

Экспортированный по умолчанию элемент проще импортировать.

example_8. module/mathematic.js

```
// константа pi
const pi = Math.PI;
// квадрат числа
const fnSquare = (x) => {
    return x * x;
};
```

```
// куб числа
var fnCube = (x) => {
    return x * x * x;
};
// площадь круга
let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};
// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
// экспорт дефолтного значения
export default fnAreaCircle;
```

example_8. example_8.html

```
<script type="module">
    // импортируем функцию по умолчанию
    // фигурные скобки не используются
    import fnAreaCircle from "./module/mathematic.js";
    let out = 'Площадь круга: ' + fnAreaCircle(4);
    alert(out);
</script>
```

Еще один пример использования дефолтного ...

example_9. **noname.js**

```
// экспортируем функцию без имени
export default (x) => {
    return x * x * x;
};
```

example_9. **example_9.html**

```
<script type="module">
    // импортируем функцию по умолчанию
    // фигурные скобки не используются
    // импортируемая из модуля функция не имеет имени
    import cube from "./noname.js";
    console.log(cube(x));
</script>
```

Смешанный export / import значений

Экспорт **по умолчанию** может сочетаться с **обычным** экспортом, а экспортировать элементы можно по отдельности:

example_10. **mathematic.js**

```
// константа pi
const pi = Math.PI;
// квадрат числа
const fnSquare = (x) => {
    return x * x;
};
```

```
// куб числа
var fnCube = (x) => {
    return x * x * x;
};
// площадь круга
let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};
// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};
// экспорт дефолтного значения (функции)
export default fnAreaCircle;
// экспорт константы pi
export { pi };
// экспорт функции
export { fnAreaTriangle };
```

example_10. **example_10.html**

```
<script type="module">
    // импортируем функцию по умолчанию
    // фигурные скобки не используются
    import fnAreaCircle, {pi, fnAreaTriangle} from
```

```
    "./module/mathematic.js";
    let target = document.getElementById("target");
    let out = '<p>Площадь круга: ' + fnAreaCircle(4);
    target.innerHTML = out;
    out = 'Число PI: ' + pi;
    target.insertAdjacentHTML('afterbegin', out);
    out = '<p>Площадь треугольника: ' + fnAreaTriangle(4, 2);
    target.insertAdjacentHTML('beforeend', out);
</script>
```

Особенности модулей

Всегда "use strict"

В модулях всегда используется режим **use strict**. Например, присваивание к необъявленной переменной вызовет ошибку.

```
<script type="module">
     x = 5; // ошибка
     // let x = 5; // правильно так
</script>
```

Своя область видимости переменных и функций

Каждый модуль имеет свою собственную **область видимости**. Переменные и функции, объявленные в модуле, не видны в других скриптах.

В демонстрационном примере **example_11** переменная **user**, объявленная в одном скрипте, не видна в другом. В итоге генерируется ошибка:

example_11. **example_11.html**

```
<script type="module">

    // переменная доступна только в этом модуле

    let user = "John";

</script>

<script type="module">

    alert(user); // Uncaught ReferenceError: user is not defined

</script>
```

Если нам нужно сделать глобальную переменную уровня всей страницы, можно явно присвоить её объекту **window**, тогда получить значение переменной можно, обратившись к **window.user**. Но это должно быть исключением, требующим веской причины. Модули должны экспортировать / импортировать функциональность, предназначенную для использования извне.

В следующем демонстрационном примере внешний и внутренний скрипт используют функцию с **одинаковым именем**. В таких случаях импортируемой функции можно задать **алиасное** (альтернативное) **имя**:

```
import {fnAreaCircle as ModfnAreaCircle, fnSquare, pi} from "./mathematic.js";
```

example_12. **mathematic.js**

```
// константа pi

const pi = Math.PI;

// квадрат числа

const fnSquare = (x) => {

    return x * x;

};

// куб числа

var fnCube = (x) => {
```

```
    return x * x * x;
};

// площадь круга
let fnAreaCircle = (r) => {
    let res = pi * fnSquare(r);
    res = res.toFixed(2);
    return res;
};

// площадь треугольника
function fnAreaTriangle (a, h) {
    return (a * h) / 2;
};

// экспорт нескольких значений
export { fnAreaCircle, fnSquare, pi };
```

example_12. **example_12.html**

```
<script type="module">
    import {fnAreaCircle as ModfnAreaCircle, fnSquare, pi} from
    "./mathematic.js";
    // создадим функцию с дублирующим именем
    let fnAreaCircle = (r) => {
        let res = pi * fnSquare(r);
        return res;
    };
    let out = 'Площадь круга: ' + ModfnAreaCircle(4);
    out += '<p>Площадь круга: ' + fnAreaCircle(4);
```

```
    let target = document.getElementById("target");
    target.insertAdjacentHTML('beforeend', out);
</script>
```

Код в модуле выполняется только один раз при импорте

Если один и тот же модуль используется в нескольких местах, то его код **выполнится только один раз**, после чего экспортируемая функциональность передаётся всем импортёрам.

Это очень важно для понимания работы модулей. Разберем следующий пример. В модуле **user.js** создается, инициализируется начальными значениями и экспортируется объект **user**.

example_13. **user.js**

```
// создаем и экспортируем объект
export let user = {
    login: null,
    email: null,
    role: null
}
// инициализируем свойства объекта начальными значениями
user.login = "NoName";
user.email = "No email address";
user.role = "user";
```

В модуле **user-ini.js** переопределяем свойства объекта.

example_13. **user-ini.js**

```
// импортируем объект user
```

```
import { user } from './user.js';
// переопределяем свойства объекта
user.login = prompt("Введите значение login:");
user.email = prompt("Введите значение email:");
user.role = "SuperAdmin";
```

Теперь, при повторном подключении объекта **user**, будут ли сброшены его свойства к начальным значениям?

example_13. **example_13.html**

```
<!-- модуль user-ini подключает объект user и переопределяет его свойства -->
<script type="module" src="./user-ini.js"></script>
<!-- еще раз подключаем объект user -->
<script type="module">
    import { user } from "./user.js";
    let per = `
        <h3>Информация о пользователе</h3>
        login: ${user.login}<br>
        email: ${user.email}<br>
        role: ${user.role}
    `;
    document.getElementById("user").innerHTML = per;
</script>
<div id="user"></div>
```

В приведенном примере видно, код модуля **user.js** выполнился только один раз.

Если вы не обратили внимание, в демо-примере **example_13** скрипт находится выше целевого блока вставки генерируемого контента:

```
<div id="user"></div>
```

Тем не менее **вставка выполнена без проблем**. Об этом далее.

Особенности браузеров

Есть и несколько других, именно браузерных особенностей скриптов с **type="module"** по сравнению с обычными скриптами.

Модули являются отложенными (deferred)

Модули всегда выполняются в **отложенном** (**deferred**) **режиме** точно так же, как скрипты с атрибутом **defer**. Это верно и для **внешних,** и для **встроенных** скриптов-модулей.

Другими словами:

- загрузка внешних модулей, таких как <script type="module" src="...">, **не блокирует обработку HTML**;
- модули, даже если загрузились быстро, **ожидают полной загрузки HTML документа**, и только затем выполняются.
- сохраняется относительный порядок скриптов: скрипты, которые идут раньше в документе, выполняются раньше.

Модули всегда видят полностью загруженную HTML-страницу, включая элементы под ними.

Поведение скриптов наглядно продемонстрировано в следующем примере:

example_14. **citate.js**

```
// сообщение пользователю

let citate = 'Будьте внимательны к своим мыслям, они — начало
```

```
поступков (Лао Цзы).';

// вывод в консоль значения цитаты

console.log("Цитата дня: ", citate);
```

example_14. example_14.html

```
<!-- сначала подключаем модуль -->

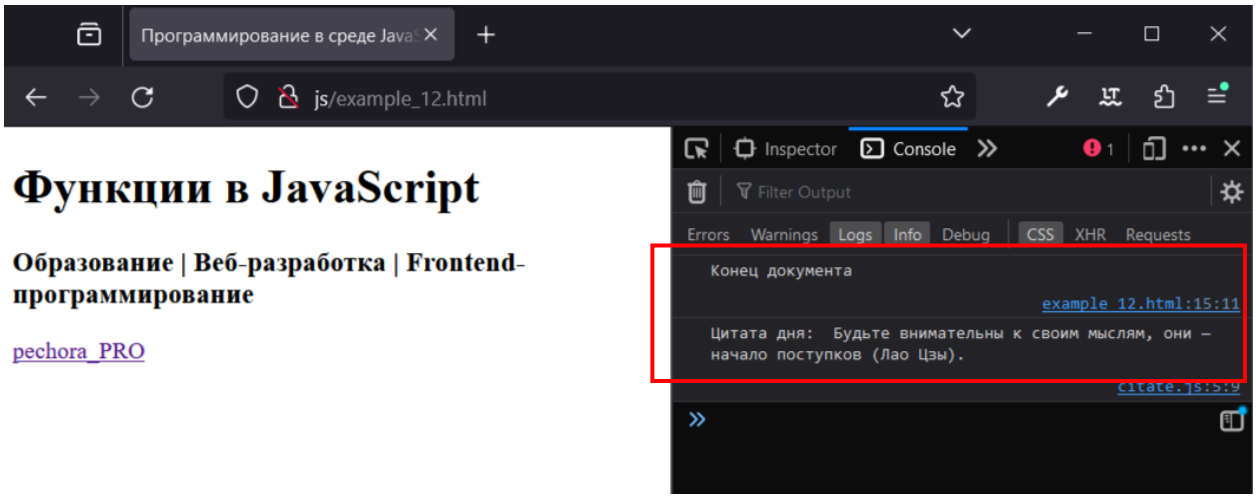
<script type="module" src="./citate.js"></script>

<!-- затем внедряем скрипт -->

<script>

    console.log('Конец документа');

</script>
```



Важно. Модули ожидают **полной загрузки** HTML документа, и только затем **выполняются**.

Не допускаются «голые» модули

В браузере **import** должен содержать относительный или абсолютный путь к модулю. Модули без пути называются *голыми* (bare). Они не разрешены в **import**.

Например, такой import неправильный:

```
import {sayHi} from 'sayHi.js'; // ошибка, "голый" модуль

// путь должен быть, например: './sayHi.js'
```

Другие окружения, например **Node.js**, допускают использование «голых» модулей, без путей, так как в них есть свои правила, как работать с такими модулями и где их искать.

Немного практики

На самом деле модули — это очень удобно. Привыкнув к работе с модулями, вы уже не сможете отказать себе в удовольствии оформить код именно таким образом.

В демонстрационном примере **example_15** выполним первый шаг к такой работе. В файле **teams.js** определим и экспортируем трехмерный массив.

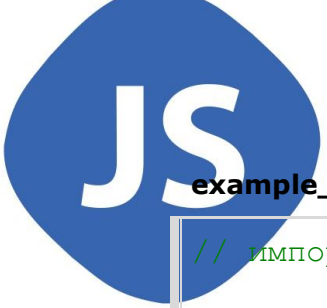
example_15. **teams.js**

```
// определим и экспортируем трехмерный массив
export default [
    [
    1, "Aerosmith", "aerosmith", "США", "Группа Aerosmith ...",
    1970, ["хард-рок", "блюз-рок", "фолк"],
    https://ru.wikipedia.org/wiki/Aerosmith
    ],
    [
    2, "Pink Floyd", "pink-floyd", "Великобритания", "Группа Pink
    Floyd ...", 1965, ["психоделический-рок", "арт", "фолк-рок"],
```

```
    https://ru.wikipedia.org/wiki/Pink_Floyd
    ],
    [
    3, "The Beatles", "the-beatles", "Великобритания", "Группа
    The Beatles ...", 1960, ["рок-н-ролл", "поп", "блюз"],
    https://ru.wikipedia.org/wiki/The_Beatles
    ],
    [
    4, "AC/DC", "ac-dc", "Австралия", "Группа AC/DC ...", 1973,
    ["хард-блюз-рок", "рок-н-ролл", "рок"],
    https://ru.wikipedia.org/wiki/AC/DC
    ],
    [
    5, "Scorpions", "scorpions", "ФРГ", "Группа Scorpions ...",
    1965, ["хард-рок", "поп-рок", "рок"],
    https://ru.wikipedia.org/wiki/Scorpions
    ],
    [
    6, "Ленинград", "ленинград", "Россия", "Группа Ленинград
    ...", 1997, ["ска", "фолк", "панк"],
    https://ru.wikipedia.org/wiki/%D0%9B%D0%B5%D0%BD%D0%B8%D0%BD%
    D0%B3%D1%80%D0%B0%D0%B4_(%D0%B3%D1%80%D1%83%D0%BF%D0%BF%D0%B0
    )
    ]
];
```

Файл **functions.js** является модулем, определяющим функции логики приложения:

- **fnGetData**() – функция получения данных;
- **fnOutList**() – вспомогательная функция вывода списка.



example_15. functions.js

```
// импортируем массив
import teams from './teams.js';
// определяем и экспортируем функцию получения данных
// передаем параметр по умолчанию - импортированный массив teams
export function fnGetData(data=teams) {
    let res = '';
    // перебираем элементы массива teams
    for (let i=0; i<data.length; i++) {
        // для вложенного массива стилей группы
        // -> вызываем вспомогательную функцию вывода списка
        let list = fnOutList(data[i][6]);
        res += `
            <h3>${data[i][1]}</h3>
            <ol>${list}</ol>
        `;
    }
    // возвращаем результат
    return res;
};
// определяем вспомогательную функцию вывода списка
function fnOutList(arr) {
    let res = '';
    for (let i=0; i<arr.length; i++) {
        res += '<li>' + arr[i] + '</li>';
    };
```

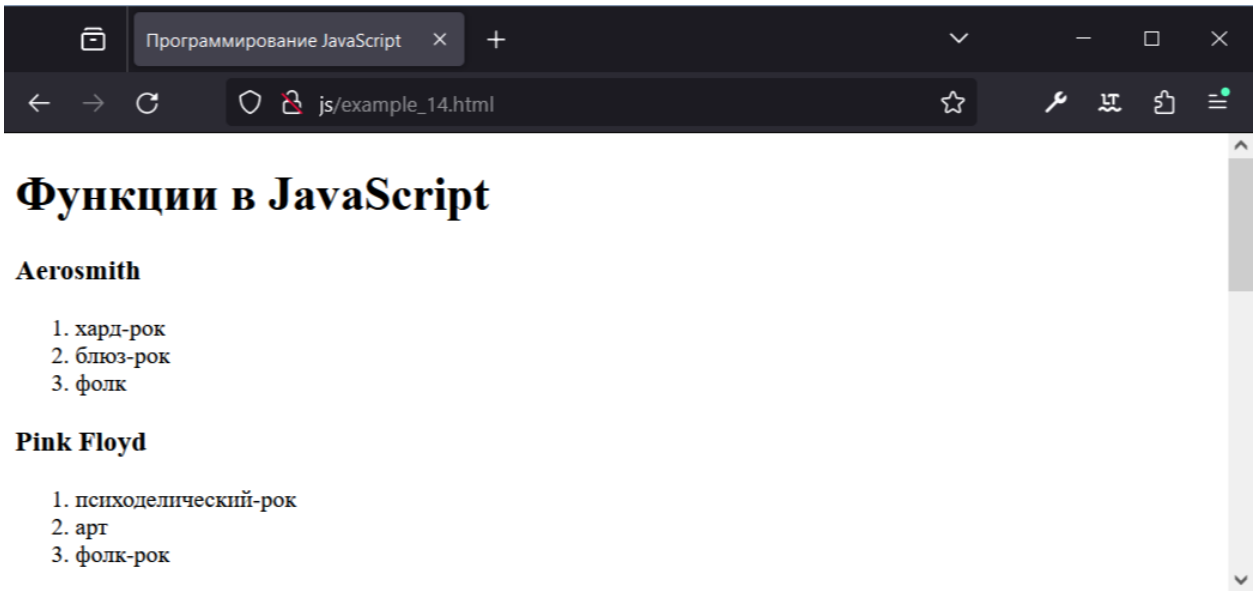
```
    // возвращаем результат
    return res;
}
```

Файл **example_15.html** является стартовым файлом приложения. Именно в нем запускается механизм запуска функций модуля.

example_15. example_15.html

```
<script type="module">
    import { fnGetData } from "./functions.js";
    // вызываем импортированную функцию списка
    let html = fnGetData();
    // без всяких блоков-контейнеров вставляем сформированный
    контент в браузер
    document.body.firstElementChild.insertAdjacentHTML("afterend"
    , html);
</script>
```

Примерный **формат** вывода приложения представлен на скриншоте.



Больше примеров и практики в **самостоятельных работах**:

- Вывод блока footer из функции внешнего модуля.
- Расчет корней квадратного уравнения с помощью функций внешнего модуля.
- Взаимодействие модуля с деревом DOM документа.
- Загрузка в модуль внешних данных. Взаимодействие модулей.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.